

EXPERIMENT-9

ROLL NO: 240701058

NAME: ASHWIN C

USER INTERFACE AND DESIGN

AIM:

The aim is to design a registration form that validates data such as name, email, phone number, and passwords, and displays error messages using HTML/CSS and JavaScript.

PROCEDURE:

Step 1: Setting Up the HTML Form

Create an HTML structure for an "Input Form Validation" containing input fields for Full Name, Email Address, Phone Number, Password, and Confirm Password.

HTML (index.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Input Form Validation</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="page-shell">
    <section class="form-panel" aria-labelledby="form-title">
      <div class="form-copy">
        <p class="eyebrow">Login</p>
        <h1 id="form-title">Input Form Validation</h1>
      </div>
      <form id="registrationForm" class="validation-form" novalidate>
        <div class="field-group">
          <label for="fullName">Full Name</label>
          <input type="text" id="fullName" name="fullName"
            placeholder="Enter your full name"
            autocomplete="name" aria-describedby="fullNameError">
          <span class="error-message" id="fullNameError"></span>
        </div>
        <div class="field-group">
          <label for="email">Email Address</label>
          <input type="email" id="email" name="email"
            placeholder="name@example.com"
            autocomplete="email" aria-describedby="emailError">
          <span class="error-message" id="emailError"></span>
        </div>
      </form>
    </section>
  </main>
</body>
</html>
```

```

    </div>
    <div class="field-group">
      <label for="phone">Phone Number</label>
      <input type="tel" id="phone" name="phone"
        placeholder="+91 98765 43210"
        autocomplete="tel" aria-describedby="phoneError">
      <span class="error-message" id="phoneError"></span>
    </div>
    <div class="field-group">
      <label for="password">Password</label>
      <input type="password" id="password" name="password"
        placeholder="Minimum 8 characters"
        autocomplete="new-password"
        aria-describedby="passwordError">
      <span class="error-message" id="passwordError"></span>
    </div>
    <div class="field-group">
      <label for="confirmPassword">Confirm Password</label>
      <input type="password" id="confirmPassword"
        name="confirmPassword"
        placeholder="Re-enter your password"
        autocomplete="new-password"
        aria-describedby="confirmPasswordError">
      <span class="error-message"
        id="confirmPasswordError"></span>
    </div>
    <button type="submit">Submit Form</button>
    <p class="success-message"
      id="successMessage" role="status"></p>
  </form>
</section>
</main>
<script src="https://cdn.jsdelivr.net/npm/validator@13.12.0/validator.min.js"></script>
<script src="script.js"></script>
</body>
</html>

```

Step 2: Styling the Form with CSS

Apply styling to center the form, define the layout of the container, and style error/success messages.

CSS (style.css):

```

* { box-sizing: border-box; }

body {
  margin: 0;
  min-height: 100vh;
  font-family: Arial, Helvetica, sans-serif;
  color: #1f2937;
  background: #eef5f1;
}

.page-shell {
  min-height: 100vh;
  display: grid;
  place-items: center;
}

```

```
padding: 32px 16px;
}

.form-panel {
width: min(980px, 100%);
display: grid;
grid-template-columns: 0.9fr 1.1fr;
gap: 32px;
align-items: center;
padding: 32px;
border-radius: 8px;
background: #ffffff;
box-shadow: 0 24px 70px rgba(31, 41, 55, 0.14);
}

.form-copy { padding: 16px; }

.eyebrow {
margin: 0 0 12px;
font-size: 13px;
font-weight: 700;
text-transform: uppercase;
letter-spacing: 0.08em;
color: #047857;
}

h1 {
margin: 0 0 16px;
font-size: clamp(32px, 5vw, 52px);
line-height: 1.05;
color: #0f172a;
}

.validation-form { display: grid; gap: 18px; }

.field-group { display: grid; gap: 7px; }

label {
font-size: 14px;
font-weight: 700;
color: #111827;
}

input {
width: 100%;
min-height: 48px;
border: 1px solid #cbd5e1;
border-radius: 6px;
padding: 12px 14px;
font-size: 16px;
color: #111827;
background: #ffffff;
outline: none;
transition: border-color 0.2s ease, box-shadow 0.2s ease;
}

input:focus {
border-color: #047857;
box-shadow: 0 0 0 4px rgba(4, 120, 87, 0.14);
}

input.is-invalid {
```

```

    border-color: #dc2626;
    background: #fff7f7;
}

input.is-valid { border-color: #059669; }

.error-message {
    min-height: 18px;
    font-size: 13px;
    line-height: 1.4;
    color: #b91c1c;
}

button {
    min-height: 50px;
    border: 0;
    border-radius: 6px;
    padding: 12px 18px;
    font-size: 16px;
    font-weight: 700;
    color: #ffffff;
    background: #047857;
    cursor: pointer;
    transition: background 0.2s ease, transform 0.2s ease;
}

button:hover { background: #065f46; }

button:active { transform: translateY(1px); }

.success-message {
    min-height: 22px;
    margin: 0;
    font-size: 14px;
    font-weight: 700;
    color: #047857;
}

@media (max-width: 760px) {
    .form-panel {
        grid-template-columns: 1fr;
        padding: 22px;
    }
}

```

Step 3: Adding JavaScript for Validation

Write a script to check if the name is empty and at least 3 characters, ensure the email is valid, validate the phone number format, verify password strength (8+ characters with uppercase, lowercase, number, and symbol), and ensure passwords match.

JavaScript (script.js):

```

const form = document.getElementById("registrationForm");
const successMessage = document.getElementById("successMessage");

```

```

const fields = {
  fullName: document.getElementById("fullName"),
  email: document.getElementById("email"),
  phone: document.getElementById("phone"),
  password: document.getElementById("password"),
  confirmPassword: document.getElementById("confirmPassword")
};

const errorMessages = {
  fullName: document.getElementById("fullNameError"),
  email: document.getElementById("emailError"),
  phone: document.getElementById("phoneError"),
  password: document.getElementById("passwordError"),
  confirmPassword: document.getElementById("confirmPasswordError")
};

const validation = window.validator || {
  isEmpty(value) { return value.length === 0; },
  isLength(value, options) { return value.length >= options.min; },
  matches(value, pattern) { return pattern.test(value); },
  isEmail(value) { return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value); },
  isMobilePhone(value) { return /^[0-9]{10,15}$/.test(value); },
  isStrongPassword(value) {
    return /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\A-Za-z0-9]).{8,}$/.test(value);
  }
};

function setFieldState(fieldName, message) {
  const input = fields[fieldName];
  const error = errorMessages[fieldName];
  const isValid = message === "";
  error.textContent = message;
  input.classList.toggle("is-invalid", !isValid);
  input.classList.toggle("is-valid", isValid && input.value.trim() !== "");
  input.setAttribute("aria-invalid", String(!isValid));
  return isValid;
}

function validateFullName() {
  const value = fields.fullName.value.trim();
  if (validation.isEmpty(value)) {
    return setFieldState("fullName", "Full name is required.");
  }
  if (!validation.isLength(value, { min: 3 })) {
    return setFieldState("fullName", "Full name must contain at least 3 characters.");
  }
  if (!validation.matches(value, /^[A-Za-z ]+$/)) {
    return setFieldState("fullName", "Full name can contain only letters and spaces.");
  }
  return setFieldState("fullName", "");
}

function validateEmail() {
  const value = fields.email.value.trim();
  if (validation.isEmpty(value)) {
    return setFieldState("email", "Email address is required.");
  }
  if (!validation.isEmail(value)) {
    return setFieldState("email", "Enter a valid email address.");
  }
  return setFieldState("email", "");
}

```

```

function validatePhone() {
  const value = fields.phone.value.trim();
  if (validation.isEmpty(value)) {
    return setFieldState("phone", "Phone number is required.");
  }
  if (!validation.isMobilePhone(value)) {
    return setFieldState("phone", "Enter a valid phone number.");
  }
  return setFieldState("phone", "");
}

function validatePassword() {
  const value = fields.password.value;
  if (validation.isEmpty(value)) {
    return setFieldState("password", "Password is required.");
  }
  if (!validation.isStrongPassword(value)) {
    return setFieldState("password",
      "Use 8+ characters with uppercase, lowercase, number, and symbol.");
  }
  return setFieldState("password", "");
}

function validateConfirmPassword() {
  const value = fields.confirmPassword.value;
  if (validation.isEmpty(value)) {
    return setFieldState("confirmPassword", "Please confirm your password.");
  }
  if (value !== fields.password.value) {
    return setFieldState("confirmPassword", "Passwords do not match.");
  }
  return setFieldState("confirmPassword", "");
}

const validators = {
  fullName: validateFullName,
  email: validateEmail,
  phone: validatePhone,
  password: validatePassword,
  confirmPassword: validateConfirmPassword
};

Object.keys(fields).forEach((fieldName) => {
  fields[fieldName].addEventListener("blur", validators[fieldName]);
  fields[fieldName].addEventListener("input", () => {
    successMessage.textContent = "";
    validators[fieldName]();
    if (fieldName === "password" && fields.confirmPassword.value !== "") {
      validateConfirmPassword();
    }
  });
});

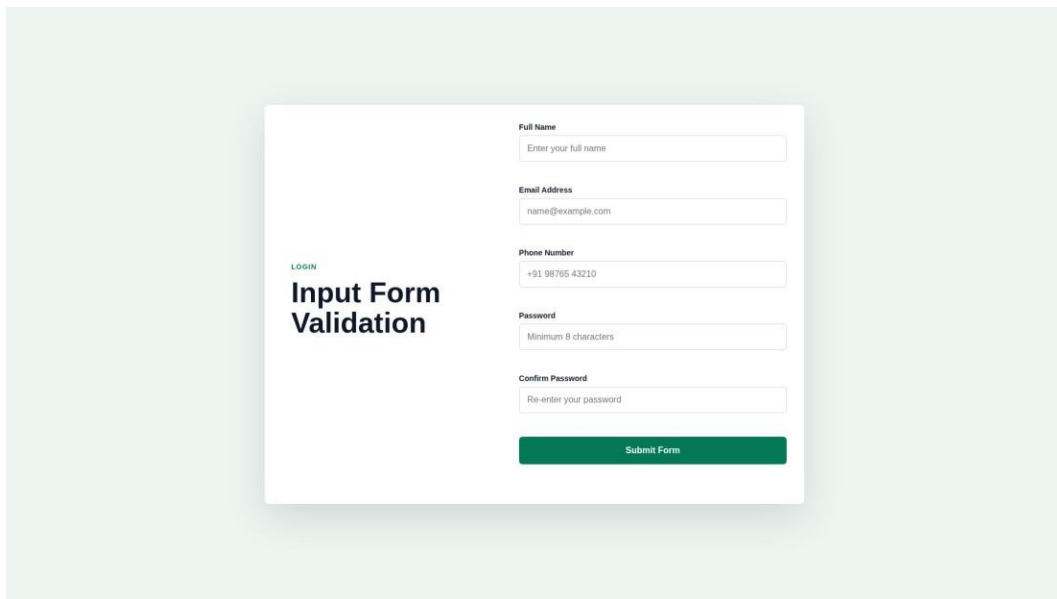
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const results = Object.values(validators).map((validate) => validate());
  const isValid = results.every(Boolean);
  if (!isValid) {
    successMessage.textContent = "";
    return;
  }
  successMessage.textContent = "Form submitted successfully.";
});

```

```
form.reset();
Object.values(fields).forEach((input) => {
  input.classList.remove("is-invalid");
  input.setAttribute("aria-invalid", "false");
});
});
```

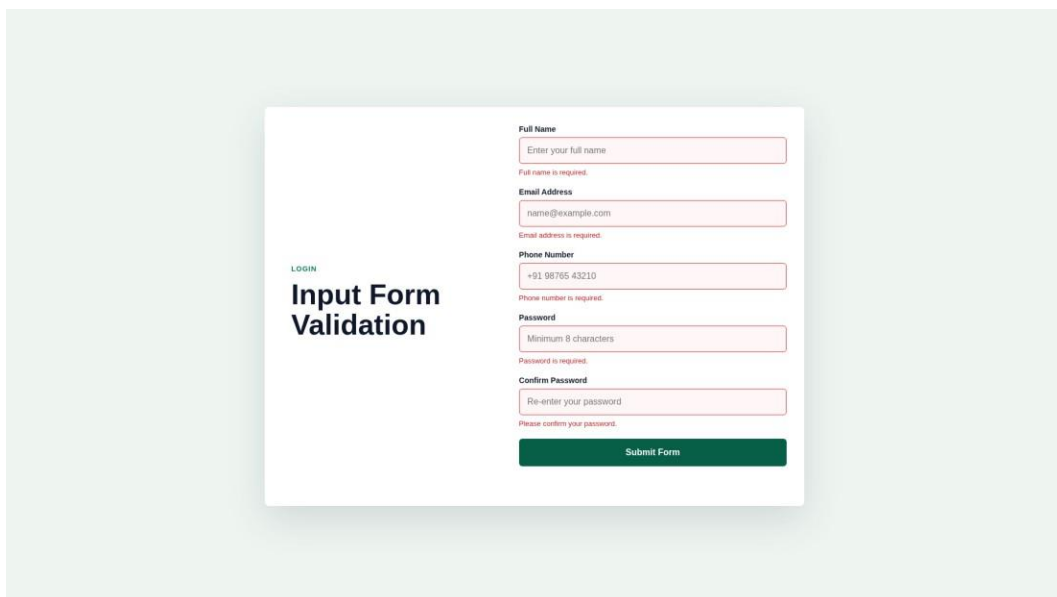
OUTPUT:

Figure 1: Registration Form - Initial State



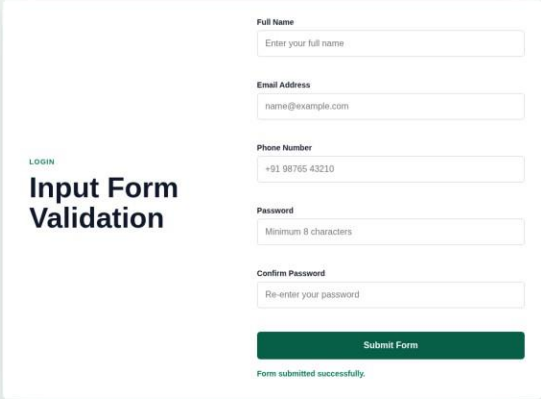
The image shows a registration form titled "Input Form Validation" with a "LOGIN" link. The form contains five input fields: "Full Name" (placeholder: "Enter your full name"), "Email Address" (placeholder: "name@example.com"), "Phone Number" (placeholder: "+91 98765 43210"), "Password" (placeholder: "Minimum 8 characters"), and "Confirm Password" (placeholder: "Re-enter your password"). A green "Submit Form" button is at the bottom right. The form is in its initial state with no validation errors.

Figure 2: Registration Form - Validation Error Messages



The image shows the same registration form as Figure 1, but with validation error messages displayed below each input field. The errors are: "Full name is required." for the Full Name field, "Email address is required." for the Email Address field, "Phone number is required." for the Phone Number field, "Password is required." for the Password field, and "Please confirm your password." for the Confirm Password field. The input fields are highlighted with a red border, and the "Submit Form" button remains green.

Figure 3: Registration Form - Successful Submission



LOGIN

Input Form Validation

Full Name
Enter your full name

Email Address
name@example.com

Phone Number
+91 98765 43210

Password
Minimum 8 characters

Confirm Password
Re-enter your password

Submit Form

Form submitted successfully.

RESULT:

The registration form was successfully designed and implemented with functional data validation for full name, email, phone number, and passwords using HTML, CSS, and JavaScript.